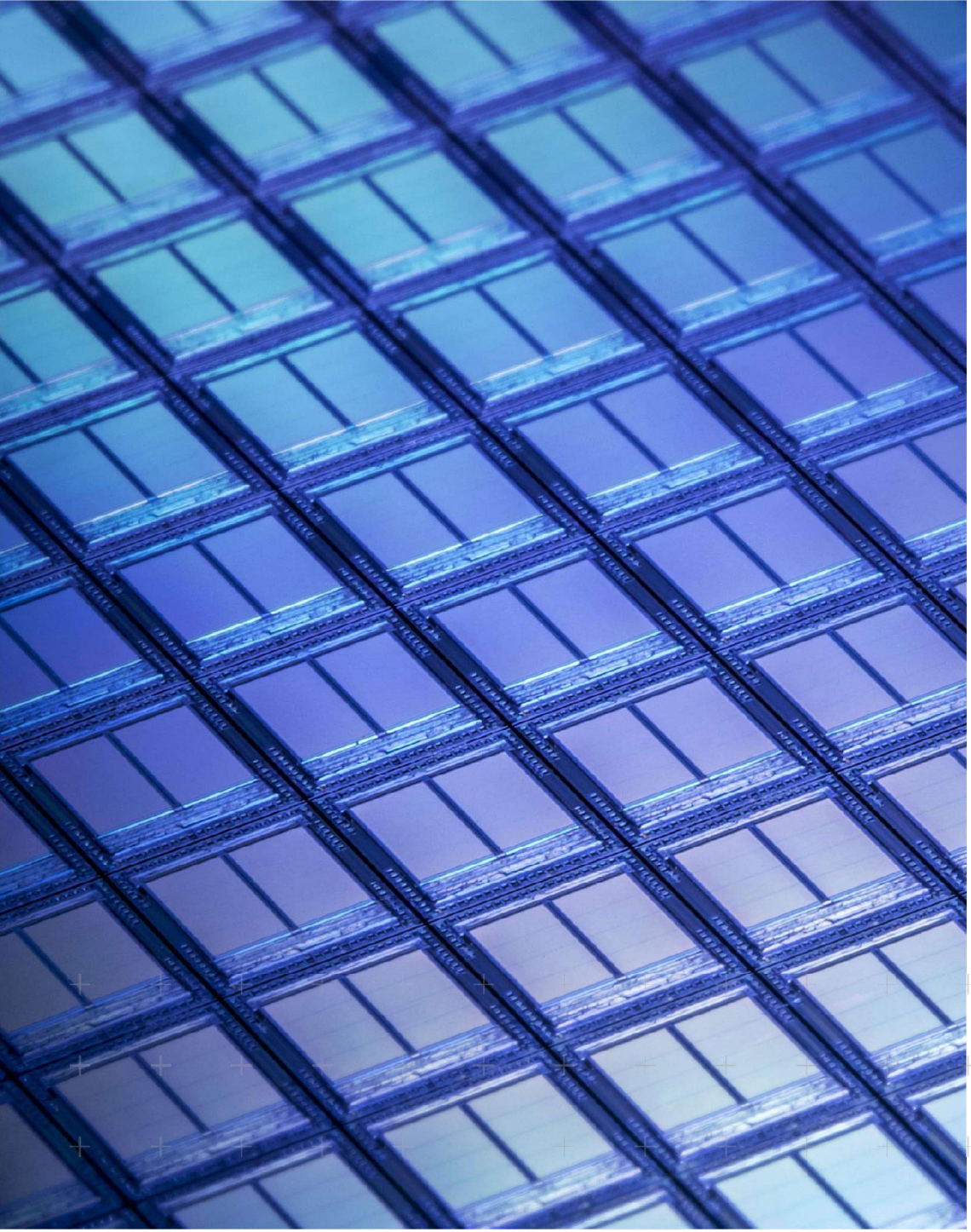

AI-Based Restor
Degraded

KLA AI Hackathon — Challe

Section 1: High-Level Overview

Challenge summary and key objectives

Challenge at a Glance

- Develop AI solutions to restore signal-degraded images
- Multiple degradation types to address, examples below:
  - Speckle noise
  - Reduction in spatial resolution
- Goal: reconstruct images to their original (or near-original) state,
  - including restoration of full spatial resolution

Why This Matters

- Image quality is critical in precision measurement and inspection
- Signal degradation causes measurable information loss
  - Histograms of degraded vs. ground truth images reveal intensity spread
  - Speckle noise can push pixel values beyond the ground truth range
- AI-driven restoration enables:
  - Recovery of fine spatial detail lost during downsampling
  - Suppression of speckle noise across diverse image types
  - Broad generalization across multiple image distributions
- Solutions must be both accurate and computationally efficient

Sample Training Data — Figure 1

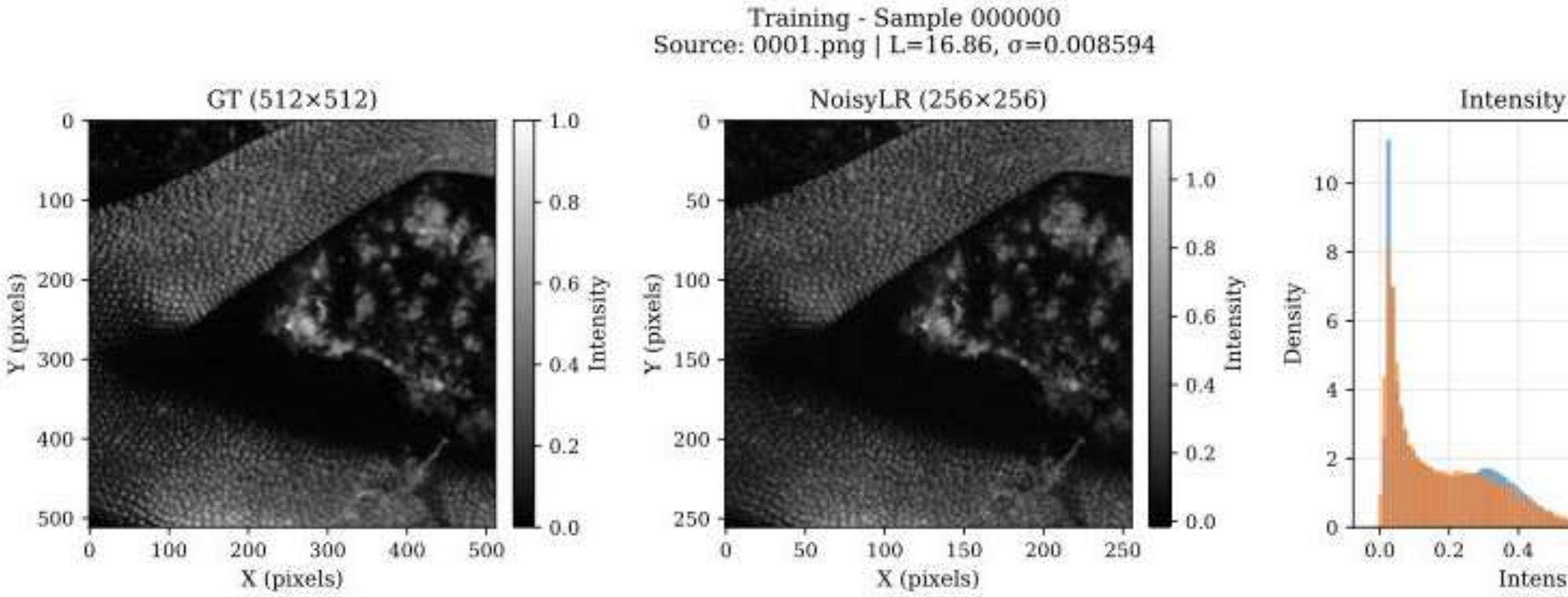

Figure 1: (Left) Ground truth 512×512 image. (Middle) Low-resolution noisy image 256×256. (Right) Intensity histograms showing inf
degradation. Note: NoisyLR intensity range exceeds ground truth due to speckle noise.

Dataset — Quick Look

- Training set: paired degraded + ground truth images
  - Ground truth resolution: 512×512 pixels or 256×256 pixels
  - Degraded images: lower resolution (256×256) or (128x128) due to down sampling 512->256, 256->
  - Diverse data origins to support generalization
- Test set (released later):
  - Includes in-distribution samples (similar to training)
  - Includes out-of-distribution samples from different sources
  - Evaluates both accuracy and robustness

Evaluation — Quick Summary

- Submissions scored on two dimensions:
  - Restoration quality (image reconstruction metrics)
  - End-to-end inference time on NVIDIA H100 GPU
- Quality metrics:
  - Structural Similarity Index Metric (SSIM)
  - Peak Signal-to-Noise Ratio (pSNR)
  - Learned Perceptual Image Patch Similarity (LPIPS)
- Faster, well-optimized pipelines preferred when quality is comparable

Section 2: Detailed Problem Statement

Dataset, evaluation criteria, and submission requirements

Problem Statement

- Develop robust AI solutions capable of restoring signal-degraded images
- Degradation types to address:
  - Speckle noise — pixel-level noise that can exceed ground truth range
  - Gausian noise — reduction in image sharpness and edge detail
  - Spatial resolution reduction — downsampling from 512×512 to 256×256 pixels or 256x256 to 128x
- Participants must design AI-driven methodologies that:
  - Effectively reconstruct and restore signal in degraded images
  - Restore spatial resolution to 512×512 pixel or 256x256 pixel ground truth

Dataset — Training Set

- Paired dataset: degraded image + ground truth image for each sample
- Designed to facilitate supervised learning approaches
- Ground truth images: 512×512 pixel or 256x256 pixel (high resolution, high SNR)
- Degraded images: lower resolution (256×256 and 128x128) due to downsampling 512->256
  - Note: degraded image intensity range may exceed ground truth range
  - due to speckle noise addition — this is expected behavior
- Diverse data origins: models should generalize across
  - multiple image categories and distributions

Training Sample — Figure 1 (Texture)

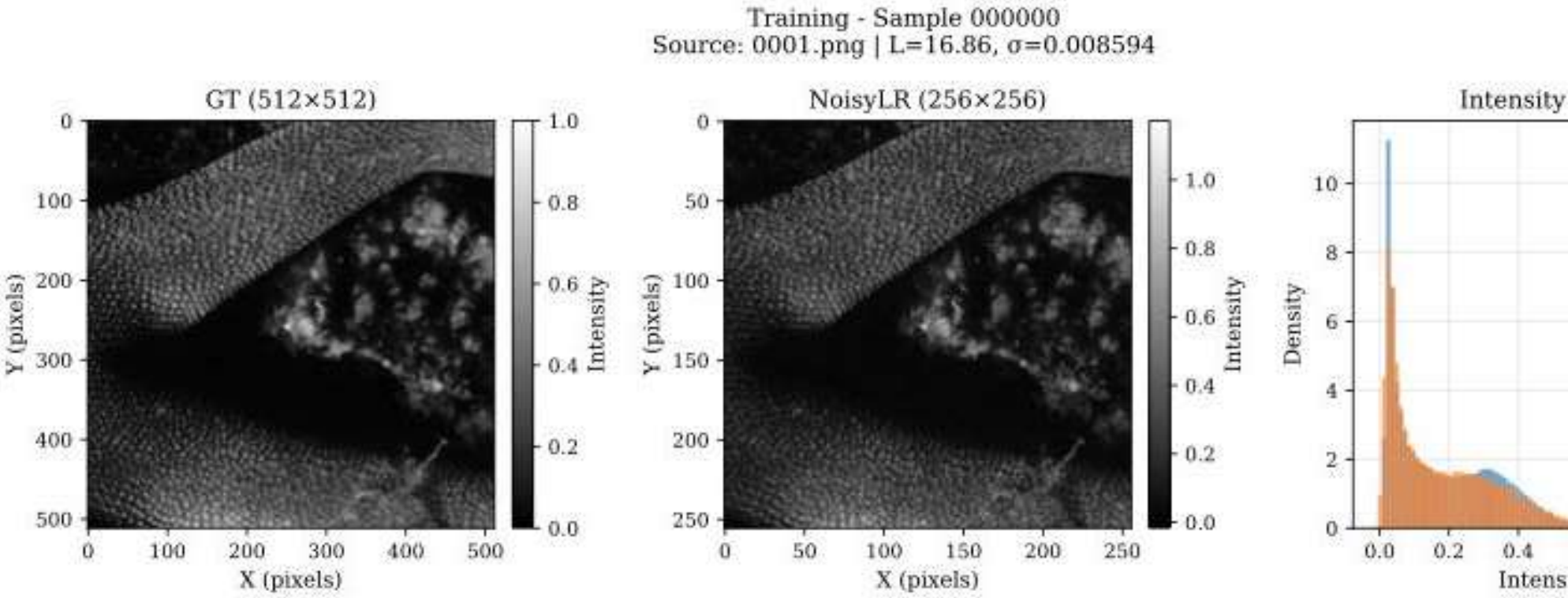

Figure 1: Ground truth GT (512×512) vs. NoisyLR (256×256) for a texture sample. Histograms (right) show the wider intensity spread c due to speckle noise.

Dataset — Test Set

- Test set will be shared after the training phase
- Two types of samples included:
  - In-distribution: similar to training data
  - Out-of-distribution: from different sources — tests generalization
- Purpose: evaluate both accuracy and robustness
  - Models must handle images from unknown distributions
- Explore data augmentation and synthetic data generation strategies
  - to improve out-of-distribution performance

Training Sample — Figure 2 (Dendrite)

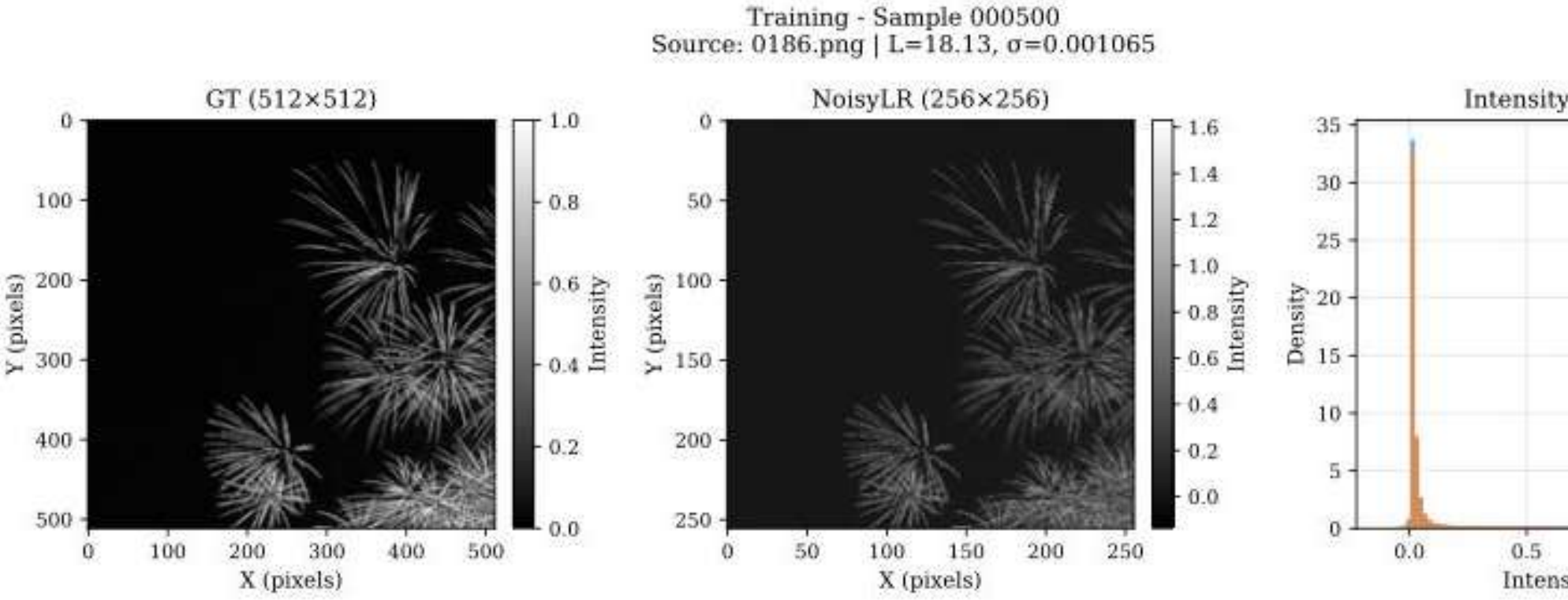

Figure 2: Ground truth GT (512×512) vs. NoisyLR (256×256) for a dendrite sample from a different data source. Histograms show signi
in the degraded image.

Evaluation — Restoration Quality Metrics

- Three standard image reconstruction metrics used for benchmarking:
  - Structural Similarity Index Metric (SSIM)
  - Measures luminance, contrast, and structural similarity
  - Peak Signal-to-Noise Ratio (pSNR)
  - Measures pixel-level fidelity between restored and ground truth images
  - Learned Perceptual Image Patch Similarity (LPIPS)
  - Measures perceptual similarity using deep features
- Participants encouraged to design effective loss functions
  - to enhance restoration quality across varying degradation scenarios

Evaluation — Inference Time Optimization

- Submissions also evaluated on end-to-end inference time
- Benchmarking system: NVIDIA H100 GPU
- Measured runtime includes:
  - Script startup and model initialization
  - Reading input images from disk
  - Performing inference on the full test set
  - Writing output images back to disk
- Teams should optimize the full inference pipeline:
  - Data loading and I/O efficiency
  - Batching and memory transfers
  - Inference execution speed
- Faster pipelines preferred when restoration quality is comparable

Section 3: Submission Details

Requirements, deliverables and participant tips

Submission Requirements

- Each submission must include four components:
- 1. Evaluation Script — standalone Python script (non-notebook)
  - Accepts: path to test images directory + path to output directory
  - Loads the trained model and runs inference on all input images
  - Writes denoised outputs to the specified directory
  - Must run without manual edits; used as-is for benchmarking
- 2. Training Script — Python script or Jupyter notebook
  - Reproduces the training of the submitted model
- 3. Denoised Test Outputs — model output on the test set
- 4. Environment Specification — complete pip freeze output
  - Required for reproducibility and benchmarking

Tips for Participants

- Prioritize model generalizability — test set includes unseen distributions
- Explore intelligent data augmentation strategies:
  - Synthetic data generation to cover additional degradation scenarios
  - Augmentation that simulates varying noise levels and noise types
- Design effective loss functions:
  - Combine perceptual loss (LPIPS) with pixel-level metrics (SSIM, pSNR)
- Optimize the full inference pipeline, not only model architecture:
  - Efficient I/O, batching, and GPU memory management matter
- Ensure your evaluation script runs end-to-end without manual intervention

Section 4: Frequently Asked Questions

Answers to common participant questions

FAQs — Dataset and Data

- Q: What resolution are the training images?
  - Ground truth: 512×512 or 256x256 pixels. Degraded images: 256×256 pixels or 128x128 pixels.
- Q: Why does the degraded image intensity range exceed the ground truth?
  - Speckle noise addition can push pixel values beyond the original signal range.
  - This is expected behavior — participants should account for it.
- Q: Can I use external data for training?
  - The problem statement does not prohibit it; synthetic data generation
  - is explicitly encouraged to improve generalization.
- Q: When is the test set released?
  - The test set will be shared later — refer to hackathon schedule.

FAQs — Evaluation

- Q: How is the final score calculated?
  - The final score considers both restoration quality (SSIM, pSNR, LPIPS)
  - and end-to-end inference time on an NVIDIA H100 GPU.
  - Faster pipelines are preferred when quality is comparable.
- Q: What does the inference time measurement include?
  - Script startup, model initialization, reading inputs from disk,
  - inference on the full test set, and writing outputs back to disk.
- Q: What GPU is used for benchmarking?
  - An NVIDIA H100 GPU using the team's provided evaluation script.
- Q: Should I optimize beyond just the model architecture?
  - Yes — data loading, batching, memory transfers, and I/O all count.

FAQs — Submission

- Q: Can I submit a Jupyter notebook as my evaluation script?
  - No. The evaluation script must be a standalone Python script (non-notebook).
  - It must run without manual edits for automated benchmarking.
- Q: What does ‘pip freeze’ environment specification mean?
  - Run ‘pip freeze > requirements.txt’ in your training environment.
  - Submit the full output to ensure reproducibility.
- Q: Must the evaluation script accept command-line arguments?
  - Yes. It must accept: (1) path to test images directory,
  - and (2) path to output directory, as inputs.
- Q: Are there restrictions on model architecture?
  - No specific restrictions — any AI-driven approach is valid.